A twelve-orbit cyclic 3-$(31, 5, 1)$ packing

Nefedov Sergey

Moscow State University, Mathematics Department

16 October 2026 Version 0.2 (working preprint)

Abstract

We exhibit twelve five-element subsets of $\mathbb{Z}_{31}$ whose full translation orbits form a cyclic 3-$(31, 5, 1)$ packing. Developing these base blocks gives 372 distinct blocks and covers 3 720 distinct triples, with none repeated. A finite certificate consisting of twelve disjoint sets of ten triple-orbit indices is included, along with independently implemented direct-shift and orbit-resource verifiers. A companion Lean 4 project encodes both finite verification routes: a direct expanded-block test using `native_decide` and a smaller resource-table check using kernel reduction. Consequently the extremal number $M(31, 5, 3)$ of base-block orbits satisfies $12 \le M(31, 5, 3) \le 13$, improving the lower bound of nine recorded in the 25 July 2026 TheoremDB research packet. We do not resolve the case $M = 13$; no impossibility certificate is claimed.

1 Problem and context

Write $G = \mathbb{Z}/31\mathbb{Z}$. For $B \in \binom{G}{5}$, let $\mathrm{Orb}(B) = \{B + t : t \in G\}$. A selection of base blocks $B_1, \dots, B_M$ is a cyclic 3-packing if every triple in $\binom{G}{3}$ belongs to at most one block in $\bigcup_i \mathrm{Orb}(B_i)$. TheoremDB's P2632 packet [1] recorded a nine-orbit construction and the upper bound $M \le 13$. The problem is equivalent to a length-31, weight-5 optical orthogonal code with off-peak auto-correlation and cross-correlation at most 2; see [2, 4] for the code setting.

Theorem 1. *There is a cyclic* 3-$(31, 5, 1)$ *packing with twelve base-block orbits. In particular,* $M(31, 5, 3) \ge 12$.

Table 1: The twelve base blocks, each written in $[0, 30]$.

i	B_i	i	B_i
1	$\{0, 1, 2, 26, 29\}$	7	$\{0, 2, 9, 13, 15\}$
2	$\{0, 1, 4, 19, 20\}$	8	$\{0, 2, 10, 14, 25\}$
3	$\{0, 1, 6, 14, 15\}$	9	$\{0, 2, 11, 21, 24\}$
4	$\{0, 1, 7, 10, 24\}$	10	$\{0, 3, 8, 11, 22\}$
5	$\{0, 1, 8, 21, 27\}$	11	$\{0, 4, 8, 13, 18\}$
6	$\{0, 2, 6, 12, 19\}$	12	$\{0, 4, 11, 16, 26\}$

2 A finite certificate

For any nonempty subset $S \subseteq G$, define its translation-canonical representative by

$$c(S) = \min_{t \in S} \mathrm{sort}\{(s - t) \bmod 31 : s \in S\},$$

where the minimum is lexicographic. The $\binom{31}{3} = 4495$ triples partition into 145 translation orbits, all of size 31: no nonzero shift can preserve a proper nonempty subset of G, because 31 is prime. Index their canonical triples in increasing lexicographic order by $0, 1, \ldots, 144$. For a base block B, write

$$R(B) = \{\text{index}(c(T)) : T \in \tbinom{B}{3}\}.$$

Each row of Table 2 specifies the indices of all ten triple orbits developed by the corresponding block in Table 1. There are *exactly 120 distinct indices* in the table (and ten per row), so the sets $R(B_i)$ are pairwise disjoint and each has cardinality ten.

Table 2: Checkable certificate: ten distinct triple-orbit indices for each B_i.

i	indices of $R(B_i)$, in increasing order
1	0, 23, 24, 26, 27, 28, 50, 52, 53, 54
2	2, 10, 11, 14, 17, 18, 65, 66, 86, 87
3	4, 12, 13, 15, 16, 21, 97, 98, 112, 113
4	5, 8, 22, 64, 71, 72, 121, 123, 126, 134
5	6, 19, 25, 79, 92, 96, 109, 116, 129, 138
6	30, 36, 43, 77, 84, 110, 111, 117, 128, 130
7	33, 37, 39, 42, 44, 51, 89, 91, 120, 122
8	34, 38, 49, 82, 88, 90, 114, 118, 133, 137
9	35, 45, 48, 57, 59, 68, 127, 140, 141, 144
10	55, 58, 61, 69, 70, 73, 103, 136, 139, 142
11	75, 76, 80, 81, 85, 94, 102, 106, 107, 135
12	78, 83, 93, 99, 100, 104, 105, 108, 131, 143

Proof of Theorem 1. The table defines twelve distinct orbits. For a single B_i, distinctness of its ten resource indices implies that no triple occurs in two different translates of B_i. Pairwise disjointness of the resource sets likewise prevents a triple from occurring in translates of two different base blocks. Translation preserves each triple orbit. Hence every triple is used at most once. Each chosen five-set has a full translation orbit of size 31. The packing therefore consists of $12 \cdot 31 = 372$ distinct developed blocks and covers $12 \cdot 31 \cdot \binom{5}{3} = 3720$ distinct triples. □

3 Upper bound and remaining gap

A fixed pair of points lies in exactly 29 triples. Every developed five-block containing that pair accounts for three distinct triples containing it. Thus the pair lies in at most $\lfloor 29/3 \rfloor = 9$ developed blocks. Counting pair–block incidences and using 31 blocks per orbit gives

$$(31M)\binom{5}{2} \le 9\binom{31}{2}, \qquad \text{so} \qquad M \le 13.$$

This recovers the previously reported bound [1, 3].

Corollary 1. *The maximum number of cyclic* 3-$(31, 5, 1)$ *base-block orbits satisfies*

$$\boxed{12 \;\le\; M(31, 5, 3) \;\le\; 13}.$$

Only the case $M = 13$ remains undecided by the present results.

For a cyclic distance $d \in \{1, \ldots, 15\}$, put $a_d(B) = \#\{\{x, y\} \subseteq B : \min(\pm(x - y) \bmod 31) = d\}$. By the same pair-incidence argument, $\sum_i a_d(B_i) \le 9$ for each d. For the displayed construction the fifteen loads are

$$(8, 7, 7, 9, 8, 8, 8, 8, 8, 8, 8, 9, 9, 8, 7).$$

These inequalities are additional cross-checks, not by themselves a complete packing certificate.

4 Enumeration and reproducibility

The resource-index verifier computes $c(T)$ for every triple of a base block. A completely separate C++ verifier instead develops all 31 translates and inserts every triple of every translated block into a set; it fails immediately upon any duplicate. Both methods confirm 372 developed blocks, 3720 distinct triples, and 775 uncovered triples out of 4495.

The full orbit enumeration gives 5481 orbits of five-sets, of which 4761 are internally admissible. The latter satisfy $|R(B)| = 10$, equivalently $|B \cap (B + d)| \leq 2$ for all $d \neq 0$. The packing problem is consequently a finite 0–1 set packing over 4761 columns and 145 resources. The twelve-orbit certificate was found by randomized neighborhood search after reproducing the known nine-orbit construction; no optimality assertion is inferred from search time, local maximality, or an incomplete MILP run.

The accompanying research bundle supplies exact input data, source code, independent verifiers, logs, a finite certificate and a SHA-256 digest. In particular, the sorted twelve-block plaintext certificate has digest

`bd23ca1859c55628896d6ff76cffe5b0db9291976fc0f7745195d44ce3d78e18`

No proof of infeasibility at thirteen orbits has been produced. Moreover, although the TheoremDB entry checked at the time of this note still lists a lower bound of nine, publication priority requires a more complete literature audit before any first-discovery claim.

5 Lean 4 formalization and verification status

For transparency and independent reproduction, we provide a companion Lean 4 project under `lean/` in the accompanying source archive. The author and affiliation are those shown on the title page. The sources pin Lean to version `v4.35.0` and use only the bundled standard library, without a Mathlib dependency.

Direct expansion certificate. The module `lean/P2632.lean` encodes the twelve base blocks of Table 1 as concrete lists of residues. It constructs all 31 modular translates per block, sorts the resulting five-sets, enumerates the ten index-selected triples in every developed block, and checks that the list of 3 720 sorted triples has no duplicates. It additionally checks that the 372 developed blocks are distinct and well formed. In the supplied source, the principal statements are `P2632.no_repeated_triple` and `P2632.exists_twelve_orbit_packing`. These larger finite decisions are expressed as `native_decide` proof obligations in the source.

Compact orbit-resource certificate. The module `lean/ResourceCertificate.lean` independently recomputes the triple-orbit index of each triple from each of the twelve base blocks and compares the results with Table 2. It checks that the table contains 120 distinct indices in the range $0, \ldots, 144$. Its named result `P2632.compact_resource_certificate` uses `decide +kernel`, invoking reduction in Lean's kernel. This module checks the finite resource data; the deduction from disjoint translation-orbit resources to the packing condition is explained in the mathematical proof above. We do not claim to have formalized that general combinatorial implication as a separate Lean theorem.

Verification methods and trust boundary. The two Lean routes have different trust models. In Lean 4, `native_decide` uses compiled native evaluation rather than a purely kernel-reduced proof. By contrast, `decide +kernel` reduces the smaller closed finite propositions in the kernel [5]. The source files include `#print axioms` commands for inspection of proof dependencies. The independently executed Python and C++ checkers provide a separate verification route for the combinatorial witness. The associated repository also contains a GitHub Actions workflow for reproducible builds.

To compile and inspect the proof dependencies with the pinned toolchain:

```
cd lean
lake build
lake env lean P2632.lean
lake env lean ResourceCertificate.lean
```

The code, literal witness, concise resource table, and verification instructions are distributed together. A successful Lean build would establish the status of the accompanying formal source, but would not resolve the global extremal question $M(31, 5, 3) = 12$ versus 13.

References

[1] TheoremDB contributors, *Largest cyclic* 3-$(31, 5, 1)$ *packing*, Problem P2632, research packet snapshot 25 July 2026. https://theoremdb.org/statements/P2632/.

[2] F. R. K. Chung, J. A. Salehi and V. K. Wei, Optical orthogonal codes: Design, analysis and applications, *IEEE Trans. Inform. Theory* **35** (1989), 595–604.

[3] R. F. Bailey and A. C. Burgess, Generalized packing designs, *Discrete Math.* **313** (2013), 1167–1190.

[4] W. Chu and C. J. Colbourn, Optimal $(n, 4, 2)$-OOC of small orders, *Discrete Math.* **279** (2004), 163–172.

[5] Lean 4 developers, *Theorem Proving in Lean 4: Tactic Reference*, sections on `decide` and `native_decide`, https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference.